SESSION 04 / PRINTABLE GUIDE

Lesson 12: Author locally in VS Code, execute on Fabric

Session 04 | Optional connected-development module
Output: A small read-only notebook edit authored in VS Code, executed on verified Fabric Spark compute, synchronized to the intended workspace, and versioned through the agreed Git route.

Explain the three separate locations

```text
VS Code / local working folder: editor and downloaded notebook content
  <-> Fabric development workspace: notebook item and remote Spark execution
  <-> Customer Git repository: reviewed source history via workspace integration
```

Local editing does not mean Spark runs on your laptop. Extension **Publish** is not Git push, a Git commit is not a run, and notebook execution is not Warehouse Gold publication. Use the same boundaries from the Git lesson.

The default exercise uses the extension's **local mode**: download, edit, select remote compute, run, and publish. The separate VFS mode edits remote workspace items directly and has different save/sync behavior; do not mix the two workflows in the first lesson.

1. Complete module-specific prerequisites before delivery

During the 3-5-business-day customer check-in, verify:

- The customer's device policy permits VS Code and the approved Microsoft **Fabric Data Engineering** extension (`SynapseVSCode.synapse`) plus the **Jupyter** extension (`ms-toolsai.jupyter`).
- The extension can authenticate against the customer's actual cloud/tenant and access the approved training workspace through its network controls. Browser access alone does not establish extension/Government-cloud support.
- Conditional Access/device restrictions and extension installation/updates are handled by the customer before class. Use an approved local folder and storage policy for notebook content and displayed/downloaded synthetic data.
- The read-only `s04_git_<pair>` practice notebook and default Lakehouse from lesson 11 are available. The first daily CSV is already staged.
- The **Microsoft Fabric Runtime** kernel/remote Spark execution path works for the intended user; Git integration is separately verified if the final version-control step is included.

Do not prescribe local PySpark, a JDK, copied access tokens, or arbitrary package installs as a workaround for a missing remote kernel. Use the approved extension/runtime prerequisites for the actual version.

If the approved cloud/sign-in authority or workspace cannot be used, mark this module **blocked** and continue the browser-based notebook lesson. Do not force a commercial-cloud endpoint, switch tenants to bypass policy, or describe a local Python run as proof of Fabric connectivity.

2. Sign in and select the correct workspace

1. Install the two approved extensions and restart VS Code if required.
2. Open the command palette and run **Fabric Data Engineering: Sign In**. Complete approved browser authentication; never paste credentials into code.
3. Inspect the signed-in account/tenant in the status bar and switch to the approved tenant if needed. Confirm it, rather than assuming the first tenant selected is correct.
4. Open the Fabric Data Engineering sidebar, choose **Select Workspace**, and select the assigned **development/training** workspace.
5. Use **Fabric Data Engineering: Set Local Work Folder** to choose an approved local working folder. For this exercise, keep it separate from a competing Git checkout; lesson 11 uses workspace Source control as the commit route.
6. Locate the pair's read-only practice notebook and use **Download**, then **Open Notebook Folder**. Alternatively, the portal's **Open in VS Code** entry can open the notebook through the extension; verify the resulting workspace/account instead of assuming they are correct.

Labels and supported commands can vary by extension version. If an operation is absent, record the version and actual behavior; do not replace a failed extension step with an unrelated API and call the lesson complete.

3. Prove remote execution before editing business logic

In the downloaded notebook, select the kernel **Microsoft Fabric Runtime**. Confirm the intended Fabric session/environment and attached default Lakehouse. A local Python interpreter or local Jupyter kernel is not an equivalent choice.

Run the practice notebook's setup and first read-only preview. Verify:

- `spark` is available on the intended Fabric runtime.
- The correct `Files/session_04/scenario/devices/readings_2026-10-01.csv` source resolves in the approved default Lakehouse.
- The notebook's execution can be observed in the actual Fabric session/run monitoring surface for the selected workspace. Record the run/session reference and runtime, without credentials.

Seeing output in VS Code, or printing a Spark version by itself, is not enough to prove that computation happened in the intended Fabric workspace. Correlate the remote kernel/context with Fabric's run evidence.

No Warehouse writes, table replacement, or local data export is needed. Keep previews bounded; a remote `toPandas()`/collect still brings records to a driver, and displayed/downloaded output can reach the developer machine.

4. Edit, compare, and publish a small read-only change

Change one Markdown explanation and add a harmless synthetic version marker. Do not alter source files, permissions, default bindings, or write flags. Use VS Code completion/help and inspect the notebook diff.

Extension state	Meaning in the documented local-mode workflow
Remote/default item	Exists in the workspace, not yet downloaded
Local (`L`)	Downloaded content matches the remote item
Modified (`M`)	Local edits have not been published to the workspace
Conflict (`C`)	Local/remote edits need deliberate resolution

Save the local edit, rerun the read-only check on the verified Fabric kernel, and review the exact target. Use **Publish** on that notebook to update the remote workspace item. Confirm the new text/marker in the Fabric portal. Coordinate with anyone who already has the item open: the portal can ask them to accept or reject an external update.

Saving locally is not publishing; publishing is not committing to Git. In the workspace Source control panel, review and commit the intended source change to the approved branch using lesson 11. Inspect the repository commit/PR separately.

5. Pull a change without losing work

With no unsaved local edits, have the presenter make a small approved Markdown-only change to the same **practice workspace item** in the browser. Use the extension's **Update Notebook** action to inspect the local/remote diff and reconcile changes. Follow the tool's merge/confirmation action before declaring synchronization complete.

If edits conflict, preserve both versions and agree the result. Do not blindly publish over another user's work, select a production workspace, or use Delete Notebook as cleanup: the extension can offer deletion of both local and remote copies. There is no deletion exercise in this lesson.

6. Diagnose the right layer

Symptom	Check first
Wrong/missing workspace	Account, tenant, workspace permission, cloud support
`spark` missing	Kernel selection/session; do not create a local Spark session as a substitute
File/table not found	Default Lakehouse and actual copied path/catalog
Works in browser, fails in VS Code	Extension identity/context, remote kernel, approved network/device policy
Local edit absent in portal	Saved versus published state, correct target, pending conflict/portal acceptance
Portal edit absent locally	Update Notebook and diff review
Git history unchanged	Publish versus workspace Commit to Git are separate steps

Record the failing layer and useful sanitized message. Do not fix permission problems by putting credentials in source or broadening access yourself.

Completion evidence

Record extension versions, approved account/tenant/workspace context, remote kernel/runtime and Fabric run reference, correct read-only preview, reviewed local diff, published portal result, reconciled incoming change, and the separate Git commit/PR if included.

The learner explains where code, compute, persisted data, and history live. Unverified cloud support, import, remote execution, publish, and Git sync remain separately labeled; one successful local operation does not prove the others.

Official references, reviewed October 29, 2026: Extension setup and local/VFS modes, local-mode notebook download/publish/remote execution, and notebook Git representation.

Source: 12_vscode_connected_authoring.md. Companion-material links open the repository originals.